



R语言在生物信息学中的应用



R基础和Markdown语法

中南大学 王诗翔

2025年

wangshx@csu.edu.cn



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



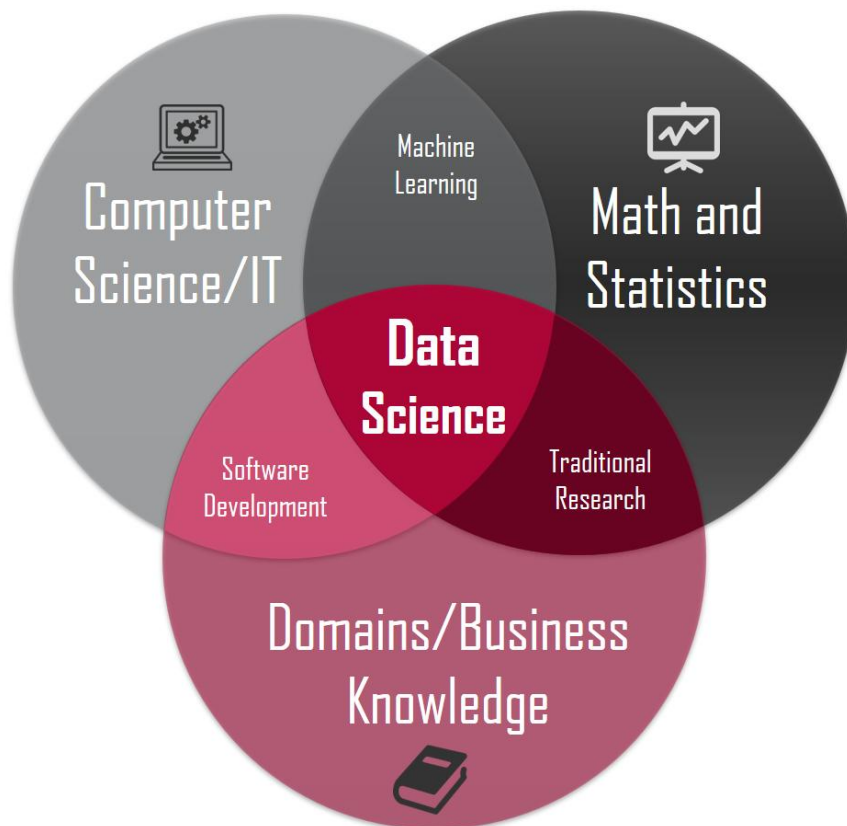
课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



1.1 什么是数据科学？

数据科学也为数学与实际应用之间建立起一个桥梁。



数据科学的韦恩图

事实上，编程是工具，统计是灵魂，专业是核心



1.2 R是什么？

1992年，新西兰奥克兰大学统计学教授 *Ross Ihaka* 和 *Robert Gentleman*，为了方便地给学生教授统计学课程，他们设计开发了R语言（他们名字的首字母都是R）。



Ross Ihaka



Robert Gentleman

创建了R语言的人

- 2000年，R1.0.0 发布
- 2004年，第一届国际useR!会议（随后每年举办一次）
- 2005年，ggplot2宏包（2018.8 - 2019.8下载量超过 1.3 亿次）
- 2012年，R2.15.2 发布
- 2013年，R3.0.2 发布，CRAN上的宏包数量5026个
- 2016年，Rstudio公司推出 tidyverse 宏包（数据科学当前最流行的R宏包）
- 2017年，R3.4.1 发布，CRAN上的宏包数量10875个
- 2019年，R3.6.1 发布，CRAN上的宏包数量15102个
- 2020年，R4.0.0 发布，CRAN上的宏包数量16054个

[The History of R](#)



1.2 R是什么？

- 官网: <https://www.r-project.org/>
- ✓ R语言是用于统计分析、图形表示和报告的编程语言
- ✓ R 是一个统计编程语言 (statistical programming)
- ✓ R 可运行于多种平台之上, 包括Windows、UNIX 和 Mac OS X
- ✓ R 拥有顶尖水准的制图功能
- ✓ R 是免费的
- ✓ R 应用广泛, 拥有丰富的库包
- ✓ 活跃社区



1.3 R大神

- 2019 年 8 月，国际统计学年会将考普斯总统奖（The Committee of Presidents of Statistical Societies Awards，简称 COPSS 奖，被誉为统计学的诺贝尔奖）奖颁给 tidyverse 的作者 **Hadley Wickham** 后，充分说明 R 语言得到了学术界的肯定和认可



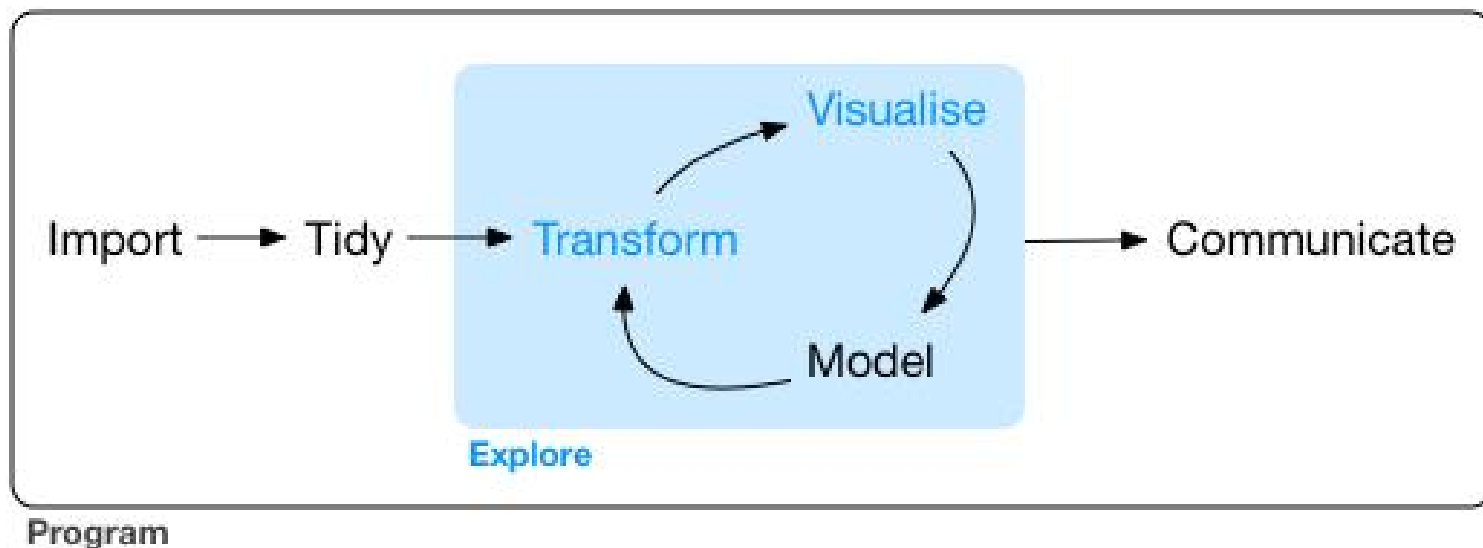
Hadley Wickham <https://hadley.nz/>

近年改变了 R 语言的人



1.4 R能干什么？

Hadley Wickham将数据科学流程分解成6个环节：



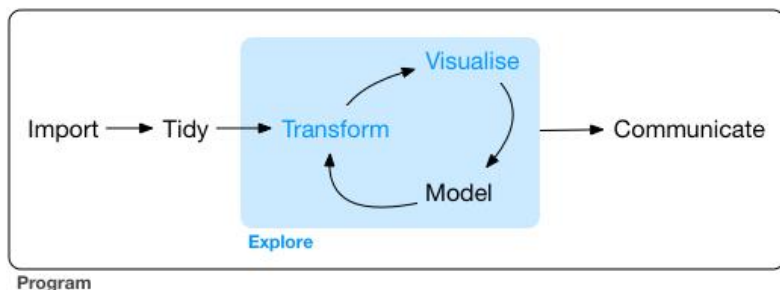
- 数据导入
- 数据规整（数据清洗）
- 数据处理
- 可视化
- 建模以及形成可重复性报告

整个分析和探索过程都在一个程序代码中完成

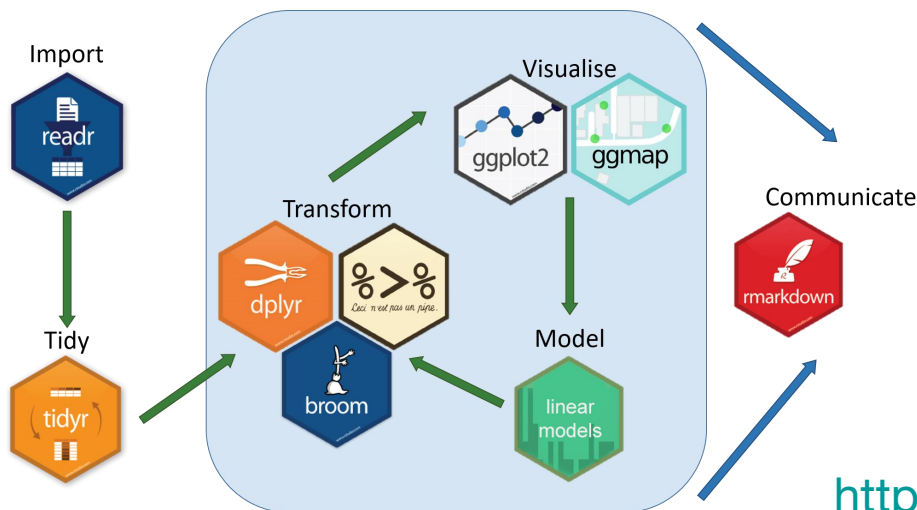


1.5 tidyverse: 数据科学全家桶

tidyverse 是一套针对数据科学设计的 R 包的集合。所有包都共享一个底层设计理念、语法和数据结构。



↓ R实现



功能

宏包

颜值担当

ggplot2

数据处理王者

dplyr

数据转换专家

tidyr

数据载入利器

readr

循环加速器

purrr

强化数据框

tibble

字符串处理

stringr

因子处理

forcats

<https://www.tidyverse.org/>



1.6 R是当今最值得学习的数据科学语言



R news and tutorials

HOME ABOUT RSS ADD YOUR BLOG! LEARN R RJOBS CONTACT US

Why R is the best data science language to learn today

Posted on January 3, 2017 by Sharp Sight Labs in R bloggers | 0 Comments

This article was first published on r-bloggers - SHARP SIGHT LABS, and kindly contributed to R-bloggers. (You can report issue about the content on this page here)

Want to share your content on R-bloggers? [click here](#) if you have a blog, or [here](#) if you don't.

[Share](#) [Tweet](#)

In last week's blog, I explained why you should Master R (even if it may eventually become obsolete).

I wrote that article to address people who claim mastering R is a bit of a waste of time (because it will eventually become obsolete).

But when I suggested that R may *eventually* become obsolete, this seemed to provoke fear that R is becoming obsolete right now.

I want to allay your fears: R is still very popular.

R has been one of the fastest growing programming languages of the last decade.

In fact, if you're getting started with data science, it's still the language that I recommend.

So, I want to reassure you. R is definitely not obsolete. In fact, R is extremely popular and a best-in class data language.

To that end, I want to explain all of the reasons why I'm very optimistic about R's long term prospects, and why I think it's perhaps **the best data science language to learn today**.

Learn frequentist statistics with R

The same can be said for statistics books.

Because R has statistics "built into its DNA," many statistics textbooks use R as a learning tool.

For an introductory look at frequentist statistics, here's one excellent book:

- Statistics: an Introduction using R

Again, if you do a quick search on Amazon, and look at many intro stats books, you'll find that if they use any programming language as a teaching tool, they are more likely to use R than almost any other language.

Learn Bayesian statistics with R

This becomes even more pronounced if you want a hands-on book for learning Bayesian statistics.

If you want to learn Bayesian stats and Bayesian analysis, nearly all of the books use R. There are some exceptions, like a few books that teach Bayesian analysis in C or Python, but overwhelmingly the best books that teach Bayesian statistics use R.

If you're interested in Bayesian stats, check out these:

- Introduction to Bayesian Statistics
- Statistical Rethinking
- Doing Bayesian Data Analysis

If you're interested in Bayesian methods, these books are "best in class," and they all use R.

Learn Data Visualization in R

When you're learning data visualization, there's a slightly larger range of programming languages to choose from, but I still maintain that most of the best learning materials use R.

If you're learning data visualization, I highly recommend the work of Nathan Yau. His book [Data Visualization with R](#) is a great resource for learning data visualization with R.

<https://www.sharpsightlabs.com/blog/r-recommend-data-science/>

数据科学，是和数据打交道（定义：用科学的方法研究数据，用数据的方法研究科学），

目的要利用（计算机和统计知识）推动学科发展，不是把大家培养成程序员。



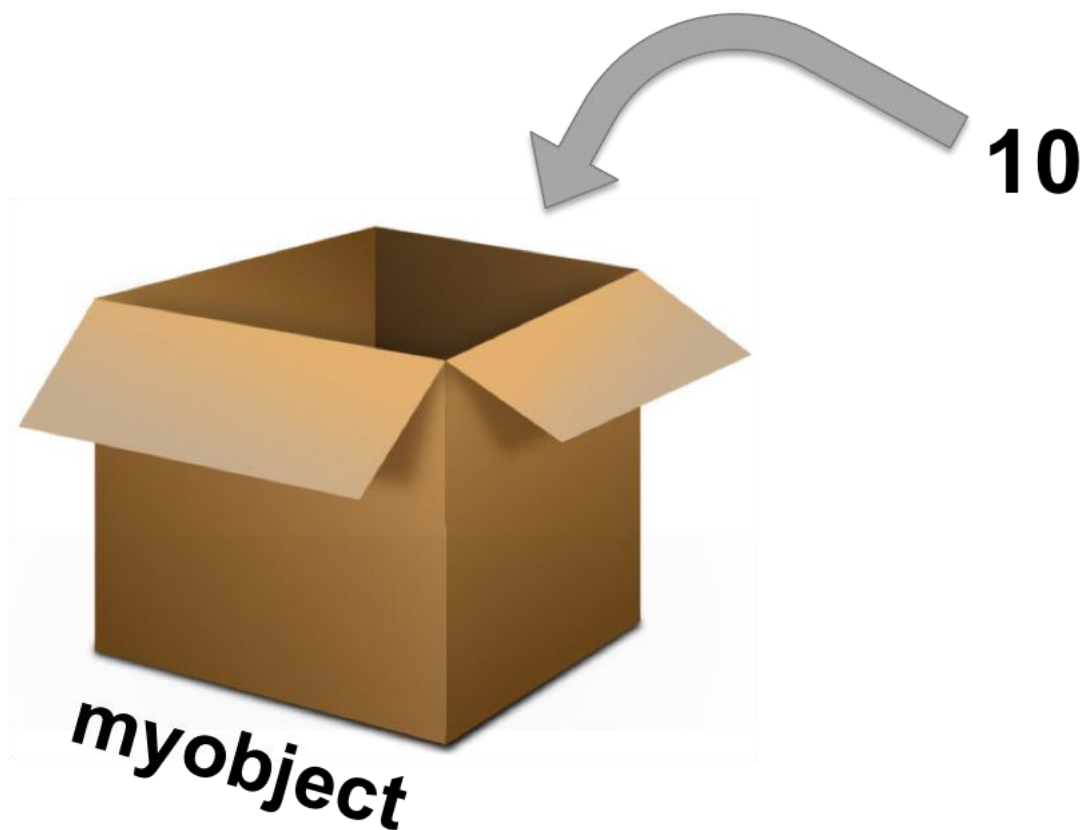
课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



2.1 一切皆对象

在R中存储的数据称为对象（object），R语言数据处理实际上就是不断的创建和操控这些对象。





2.2 R的基本运算

可以把 R 当作计算器用，比如加、减、乘、除等运算

1 + 2

5 - 3

3 * 4

12 / 3

2^3

(2 + 4) / 3

看到的都是对象!!!

怎么查看?!

- 试试**class()**函数



2.3 对象的创建和使用

快速产生赋值符号 `<-`，在Rstudio中同时按下

- `alt` 和 `-` (windows系统)
- `option` 和 `-` (苹果系统)





2.4 查看变量

当键入 `x` 然后回车，就打印出 `x` 的值。
也可以使用命令 `print(x)`，结果一样。

```
> x
[1] 5
> x+2
[1] 7
> x
[1] 5
> x <- 6
> x
[1] 6
```



2.5变量命名规则

- R变量名必须以字母、数字、下划线_和句点.组成。
- 但是，变量名的第一个字符不能为数字或者特殊符号，比如 +, -, *, /, ^, !, @, &, 或者下划线_。
- 变量名的第一个字符如果是句点., 那么句点后面不能紧跟数字。
- 变量名是区分大小写的，y和Y是两个不同的变量名。
- 在中文环境下，汉字也可以作为变量名的合法字符使用，但不推荐使用。

◆ 大小写敏感

◆ 不能有空格，可以用下划线代替空格

◆ 如 `my_age <- 30`

◆ 开头不能是数字和一些特殊符号



2.5变量命名规则

- 为了让代码可读性更强，我们希望变量名能更清晰描述所赋值的对象
 - 比如我们测量了男生的身高，并把测量结果存储为对象。

- 一般的变量名

```
x <- 175
```

- 不错变量名

```
height <- 175
```

- 最佳的变量名

```
boy_height_cm <- 175
```

第三种方案是最佳的，它清晰告诉我们男生的身高是175，测量单位是cm



2.6 对象属性

- 所有R对象都有其属性，其中最重要的两个属性是类型和长度，我们可以用以下两个命令查看对象的类型和长度

- `typeof()`

- `length()`

```
typeof(x)
```

```
## [1] "double"
```

```
length(x)
```

```
## [1] 1
```



2.7 练习

哪些是合法的变量名？

- 先 举手回答
- 再 动手试试

- `example <- c(1, 2, 3)`
- `example1 <- c(1, 2, 3)`
- `example.1 <- c(1, 2, 3)`
- `example_1 <- c(1, 2, 3)`
- `example-1 <- c(1, 2, 3)`
- `example+1 <- c(1, 2, 3)`
- `.example <- c(1, 2, 3)`
- `.2example <- c(1, 2, 3)`
- `2example <- c(1, 2, 3)`
- `_example <- c(1, 2, 3)`



课程大纲

1. 数据科学与R语言
2. R对象
- 3. R向量**
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



3.1 向量 (vector) 就像冰糖葫芦

向量是R语言最基础的数据类型。

◆ 我这里的这就像是放入盒子x, 或 concatenate 的意思

◆ 它要求元素之间用英文的逗号分隔

现在, 元素的数据类型是数据的有顺序, 这里都是数值, 比如

这样5, 6, 7) 函数把一组数据聚合到了一起, 就构成了一个向量。

```
x <- 3, 4, 5, 6, 7 # work?
```



```
x <- c(3,  
4,  
5,  
6,  
7  
)
```

vector





3.2 向量聚合

`c()` 函数还可以把两个向量聚合成一个新的向量。

```
low      <- c(1, 2, 3)
high     <- c(4, 5, 6)
sequence <- c(low, high)
sequence
```

```
## [1] 1 2 3 4 5 6|
```



3.3 命名向量

每个元素可以有自己的名字



```
x <- c('a' = 5, 'b' = 6, 'c' = 7, 'd' = 8)
x
```

```
## a b c d
## 5 6 7 8
```

或者

```
x <- c(5, 6, 7, 8)
names(x) <- c('a', 'b', 'c', 'd')
x
```

```
## a b c d
## 5 6 7 8
```



3.4 单个值是长度为1的向量

```
> x1 <- 6
```

```
> x2 <- c(6)
```

```
> identical(x1, x2)
```

```
[1] TRUE
```



3.5 数值型向量

数值型的向量，有 integer 和 double 两种：

```
x <- c(1L, 5L, 2L, 3L)      # 整数型
x <- c(1.5, -0.5, 2, 3)      # 双精度类型，常用写法
x <- c(3e+06, 1.23e2)        # 双精度类型，科学计数法
```



3.5 数值型向量

快速生成数值序列

```
s1 <- seq(from = 0, to = 10, by = 0.5)
s1
## [1] 0.0 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0
## [16] 7.5 8.0 8.5 9.0 9.5 10.0
s2 <- rep(x = c(0, 1), times = 3)
s2
## [1] 0 1 0 1 0 1
s3 <- rep(x = c(0, 1), each = 3)
s3
## [1] 0 0 0 1 1 1
s4 <- 0:10 # Colon operator (with by = 1):
s4
## [1] 0 1 2 3 4 5 6 7 8 9 10
s5 <- 10:1
s5
## [1] 10 9 8 7 6 5 4 3 2 1
```



3.6 字符串型向量

字符串（String）数据类型，实际上就是文本类型，必须用单引号或者是双引号。

```
x <- c("1", "2", "3")  
x <- c("a", "b", "c")  
x <- c('Alice', 'Bob', 'Charlie', 'Dave')  
x <- c("hello", "baby", "I love you!")
```



3.7 逻辑型向量

逻辑型常称为布尔型（Boolean），它的常量值只有 **TRUE** 和 **FALSE**。

➤ 请勿将其用作变量名。

```
x <- c(TRUE, TRUE, FALSE, FALSE)
x <- c(T, T, F, F)                # Equivalent, but not recommended

x <- c(True, False)               # good
x <- c(true, false)              # bad

x1 <- c(TRUE, FALSE)              # logical
x2 <- c("TRUE", "FALSE")         # character
```



3.8 因子型向量 (factor)

因子型可以看作是字符串向量的增强版，它是带有层级 (Levels) 的字符串向量

```
four_seasons <- c("spring", "summer", "autumn", "winter")
four_seasons

## [1] "spring" "summer" "autumn" "winter"

four_seasons_factor <- factor(four_seasons)
four_seasons_factor

## [1] spring summer autumn winter
## Levels: autumn spring summer winter

four_seasons <- c("spring", "summer", "autumn", "winter")
four_seasons_factor <- factor(four_seasons,
                             levels = c("summer", "winter", "spring", "autumn"))
four_seasons_factor
```



3.9 强制转换

矢量中的元素必须是相同的类型，但如果不一样，R会强制转换成相同的类型。

数据类型的转换层级：

- character > numeric > logical
- double > integer

```
c(1, "foo", TRUE)
## [1] "1"      "foo"    "TRUE"

c(7, TRUE, FALSE)
## [1] 7 1 0

c(1L, 2, pi)
## [1] 1.000000 2.000000 3.141593
```



3.10 向量小结

Vector	Type	Examples
	numeric	<pre>> c(1, 0.5, 3, 7) [1] 1.0 0.5 3.0 7.0</pre>
	character	<pre>> c("Alice", "love", "30", "dog") [1] "Alice" "love" "30" "dog"</pre>
	logical	<pre>> c(TRUE, FALSE, TRUE, FALSE) [1] TRUE FALSE TRUE FALSE</pre>
	factor	<pre>> factor(c("a", "c", "c", "b")) [1] a c c b Levels: a b c</pre>



3.11 练习

1. 猜猜结果是什么？

```
x <- c(1, "USA", TRUE)  
x
```

2. 要求转换成因子类型向量，并按照温度从高到低排序

```
temperatures <- c("warm", "hot", "cold")
```



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
- 4. R常见数据结构**
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



4.1 其他（基础）常见数据类型

基础的常见数据类型除了向量，还有：

- ◆ 矩阵
- ◆ 列表
- ◆ 数据框

这些数据结构都可以看作由向量衍生出来的。



4.2 矩阵 (matrix)

矩阵可以存储行(row)和列(column)二维的数据。

实际上是向量的另一种表现形式，也就说它的本质还是向量，一维的向量用二维的方式呈现。

矩阵可以用 `matrix()` 函数创建



```
matrix(  
  c(2, 4, 3, 1, 5, 7),  
  nrow = 2,  
  ncol = 3  
)
```

Matrix

2	3	5
4	1	7

```
matrix(  
  c(2, 4, 3, 1, 5, 7),  
  nrow = 2, ncol = 3,  
  byrow = TRUE )
```



?

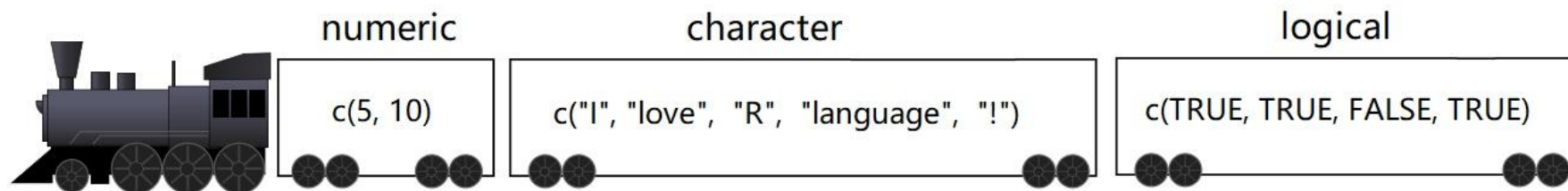


怎么查看
matrix属性



4.3 列表 (list)

想象有一个小火车，小火车的每节车厢是独立的，因此**每节车厢装的东西可以不一样**。这种结构装载数据的能力很强大，称之为**列表 (list)**



```
list(  
  a = c(5, 10),  
  b = c("I", "love", "R", "language", "!"),  
  c = c(TRUE, TRUE, FALSE, TRUE)  
)
```

List

5	"I"	TRUE
10	"love"	TRUE
	"R"	FALSE
	"language"	TRUE
	"!"	

numeric character logical



4.4 对比c()和list()

先想想？

	<code>c(a = 1, b = 2, c = 3)</code>	<pre>list(a = c(5, 10), b = c("I", "love", "R", "language", "!"), c = c(TRUE, TRUE, FALSE, TRUE))</pre>
相同点	元素之间用逗号分开。	元素之间用逗号分开。
不同点	向量的元素是单个值。	列表的元素可以是更复杂的结构， 可以是向量、矩阵或者列表。
	向量要求每个元素的数据类型必须相同， 要么都是数值型，要么都是字符型；	列表的元素允许不同的数据类型。



4.5 数据框 (data.frame)

```
data.frame(  
  name      = c("Alice", "Bob", "Carl", "Dave"),  
  age       = c(23, 34, 23, 25),  
  marriage  = c(TRUE, FALSE, TRUE, FALSE),  
  color     = c("red", "blue", "orange", "purple")  
)
```

data frame

"Alice"	23	TRUE	"red"
"Bob"	34	FALSE	"blue"
"Carl"	23	TRUE	"orange"
"Dave"	25	FALSE	"purple"

character numeric logical character

	A	B	C	D
1	name	age	marriage	color
2	Alice	23	TRUE	red
3	Bob	34	FALSE	blue
4	Carl	23	TRUE	orange
5	Dave	25	FALSE	purple

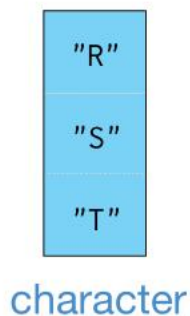
数据框类似于我们经常用的excel表格。由于数据框融合了向量、列表和矩阵的特性，所以在数据科学的统计建模和可视化中运用非常广泛。



4.6 R基础数据结构小结

可以把 R 当作计算器用，比如加、减、乘、除等运算

Vector



Matrix

"1"	"R"	"TRUE"
"2"	"S"	"FALSE"
"3"	"T"	"TRUE"

character

List



data frame

1	"R"	TRUE
2	"S"	FALSE
3	"T"	TRUE

numeric character logical



4.7 练习

创建一个（3行的）学生信息的data.frame，包含姓名、性别、年龄，成绩等变量。

Name	Sex	Age	Score
zhangsan	male	18	85
lisi	female	19	90
wangwu	male	20	88



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
- 5. 运算符与向量运算**
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



5.1 运算符

- 向量是R语言中最基础的数据结构
- R语言的强大优势就体现在向量运算上

几种运算符类型：

- ◆ 算术运算符
- ◆ 关系运算符
- ◆ 逻辑运算符
- ◆ 其他运算符



5.2 算术运算符

```
# +  
a <- c(1, 2, 3)  
b <- c(4, 5, 6)  
a + b  
## [1] 5 7 9
```

```
# -  
a - b  
## [1] -3 -3 -3
```

```
# *  
a * b  
## [1] 4 10 18
```

```
# /  
a / b  
## [1] 0.25 0.40 0.50
```

```
# %% 取余  
a %% b  
## [1] 1 2 3
```

```
# %/% 取商  
a %/% b  
## [1] 0 0 0
```

```
# %*% 两个向量的内积, 也称为点乘  
a %*% b  
## [1,] 32
```

```
# ^ 将第二向量作为第一向量的指数  
a ^ b  
## [1] 1 32 729
```



5.3 循环补齐原则

当两个向量长度不相等的时候，短的向量会循环补齐，保持与长向量的长度一致后，再做运算。

练习：

```
# 长度是整数关系
```

```
x <- 1
```

```
y <- c(10, 20, 30, 40)
```

```
x + y
```

```
x <- c(1, 2)
```

```
y <- c(10, 20, 30, 40)
```

```
x * y
```

```
# 长度不是整数关系
```

```
x <- c(1, 2, 3)
```

```
y <- c(10, 20, 30, 40, 50, 60, 70)
```

```
x + y
```

```
> x <- c(1, 2, 3)
> y <- c(10, 20, 30, 40, 50, 60, 70)
> x + y
[1] 11 11 22 33 41 52 63 71
```

警告信息：

In x + y : 长的对象长度不是短的对象长度的整倍数

长度不是整数倍关系的时候，R语言会提出友好的警告，这个警告是非常有必要的，提醒写代码的人是不是写漏了一些数据



5.4 关系运算符



```
# > 检查第一向量的每个元素是否大于第二向量的相应元素
```

```
a <- c(1, 2, 3)
```

```
b <- c(4, 5, 6)
```

```
a > b
```

```
## [1] FALSE FALSE FALSE
```

```
# < 检查第一个向量的每个元素是否小于第二个向量的相应元素
```

```
a < b
```

```
## [1] TRUE TRUE TRUE
```

```
# == 检查第一个向量的每个元素是否等于第二个向量的相应元素
```

```
a == b
```

```
## [1] FALSE FALSE FALSE
```

```
# != 检查第一个向量的每个元素是否不等于第二个向量的相应元素
```

```
a != b
```

```
## [1] TRUE TRUE TRUE
```

```
# 还有:
```

```
# >=
```

```
# <=
```



5.5 逻辑运算符

逻辑运算符，一般适用于逻辑类型的向量。

- **& 元素逻辑AND运算符**。它将第一向量的每个元素与第二向量的相应元素组合，并且如果两个元素都为TRUE，则给出输出TRUE
- **| 元素逻辑或运算符**。它将第一向量的每个元素与第二向量的相应元素组合，并且如果元素有一个为真，则给出输出TRUE。
- **! 它被称为逻辑非运算符**。对于向量的每个元素，给出相反的逻辑值。

```
a <- c(TRUE, FALSE, FALSE)
b <- c(FALSE, TRUE, FALSE)
a & b
## [1] FALSE FALSE FALSE

a | b
## [1] TRUE TRUE FALSE

!a
## [1] FALSE TRUE TRUE
```



5.6 其他运算符

: 冒号运算符。它按顺序创建一个整数序列

```
a <- 1:10
```

```
a
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

%in% 此运算符用于判断元素是否属于向量

```
c(2, 3, 7) %in% c(1, 2, 3, 4, 5)
```

```
## [1] TRUE TRUE FALSE
```

is.na() 判断是否为缺失值, R 里缺失值用NA表示, NA的意思就是 not available 或者 not applicable.

```
is.na(c(1, 2, NA, 4, NA, 6))
```



5.7 特殊值

R 语言里还有一些特殊的值: **Inf**, **NaN**, **NA** 和 **NULL**.

- **Inf**, 是Infinity的简写, 表示无限大; -Inf表示无限小, 比如我们用1除以0
- **NaN**, 是Not a Number的简写, 表示这个数字没有数学定义, 比如0除以0
- **NA**, 是Not available的简写, 表示缺失状态
- **NULL**, 是No value的意思, 表示没有值, 或者空值的意思, 表示变量实际上没有任何值, 或者甚至不存在。



5.8 练习

说出向量 a 和 b 的差异在什么地方？

```
a <- 1:10
```

```
b <- seq(from = 1, to = 10, by = 1)
```

```
identical(a, b)
```



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
- 6. R编程**
7. 函数及应用
8. 取子集
9. (R)Markdown



6.1 R编程要素

- for 循环
- while 循环
- if 语句
- 函数
- ...



流程控制结构



6.2 for 循环

对序列（如向量）元素进行依次计算处理

```
for (variable in sequence){  
    Do something  
}
```

Example

```
for (i in 1:4){  
    j <- i + 10  
    print(j)  
}
```



6.3 while 循环

循环处理直至达到退出条件

```
while (condition){  
    Do something  
}
```

Example

```
while (i < 5){  
    print(i)  
    i <- i + 1  
}
```

在R的实际应用中一般较少使用



6.4 if 语句

通过对条件判断引导程序进入不同的分支处理

```
if (condition){  
    Do something  
} else {  
    Do something different  
}
```

Example

```
if (i > 3){  
    print('Yes')  
} else {  
    print('No')  
}
```

输入?switch学习下



6.5 函数

对语句块处理的模块化实现

减少代码重复，提升日常编程的最佳途径

```
function_name <- function(var){  
    Do something  
    return(new_variable)  
}
```

Example

```
square <- function(x){  
    squared <- x*x  
    return(squared)  
}
```



6.5 其他

- repeat语句
- break/next语句
- 环境

The Environment

<code>ls()</code>	List all variables in the environment.
<code>rm(x)</code>	Remove x from the environment.
<code>rm(list = ls())</code>	Remove all variables from the environment.

You can use the environment panel in RStudio to browse variables in your environment.

```
> .GlobalEnv
<environment: R_GlobalEnv>
> ls(.GlobalEnv)
[1] "x"  "x1" "x2" "y"
```

```
repeat {
  x <- x + 1
  print(x)
  if (x >= 10) {
    break
  }
}
```

repeat循环是一个无条件循环，**必须在循环体内设置退出条件**，否则会导致无限循环。

可以使用**next**语句来**跳过当前迭代**，继续执行下一次迭代



6.6 练习

- 写一个针对常量 letters 的循环，当为m时退出
 - 使用 for/while/repeat 均可

> letters

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
[20] "t" "u" "v" "w" "x" "y" "z"
```



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
- 7. 函数及应用**
8. 取子集
9. (R)Markdown



7.1 基础函数

R 语言的强大就在于它拥有丰富的函数

➤ 这里的函数和我们高中数学中的函数 $y = f(x)$ 没什么区别。

Distributions

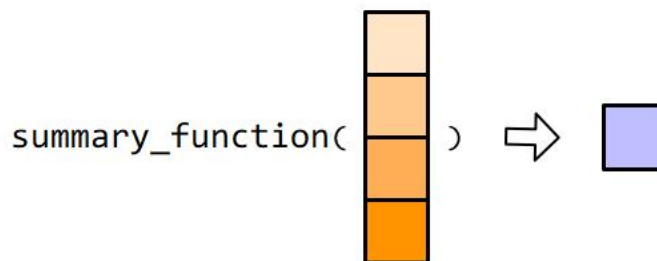
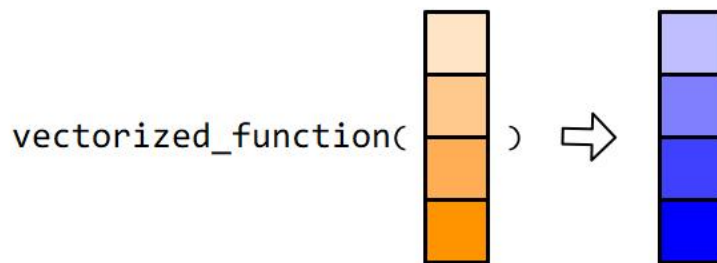
	Random Variates	Density Function	Cumulative Distribution	Quantile
Normal	<code>rnorm</code>	<code>dnorm</code>	<code>pnorm</code>	<code>qnorm</code>
Poisson	<code>rpois</code>	<code>dpois</code>	<code>ppois</code>	<code>qpois</code>
Binomial	<code>rbinom</code>	<code>dbinom</code>	<code>pbinom</code>	<code>qbinom</code>
Uniform	<code>runif</code>	<code>dunif</code>	<code>punif</code>	<code>qunif</code>



7.2 向量的函数

R 语言的函数基本都支持向量化，大大简化了代码和理解

- 用在向量上的函数，可以分为[向量化函数](#)(vectorized function)和[汇总类函数](#)(summary function)



例外：比如[unique\(\)](#)函数，它返回的向量通常不会与输入的向量等长，既不属于向量化函数，也不属于汇总类函数。



7.3 练习

```
x <- c(2, 7, 8, 9, 3)
```

```
x^2 + 5
```

```
#向量的元素减去其均值
```

```
x - mean(x)
```

```
#向量标准化（向量减去其均值之后，除以标准差）
```

```
(x - mean(x)) / sd(x)
```

如果想对更多的向量，也做标准化处理呢？

```
y <- c(1, 5, 7, 8, 9, 3)
```

```
(y - mean(y)) / sd(y)
```

```
z <- c(4, 7, 7, 8, 9, 3, 9, 6)
```

```
(z - mean(z)) / sd(z)
```

有没有有效解决重复的办法？



7.4 自定义函数

```
my_std <- function(x) {  
  (x - mean(x)) / sd(x)  
}
```

```
my_std(x)  
my_std(y)  
my_std(z)
```



7.5 练习

根据下面的数学表达式，写出函数

$$\text{rescale}(x) = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$



7.6 使用三方包函数

```
install.packages("tidyverse")
```



```
library(tidyverse)
```



```
select(starwars, height)
```





7.7 同名函数问题

如果同时加载了dplyr, MASS和skimr三个宏包, 在程序中使用select()函数, 就会造成混淆和报错。

- dplyr::select()
- skimr::select()
- MASS::select()

通过::操作符指定每个函数的来源, 进行精准调用



7.8 如何获取帮助

- ◆ 记住和学习所有的函数几乎是不可能的
- ◆ 打开函数的帮助页面(Rstudio右下面板的Help选项卡)

The screenshot displays the RStudio environment. The main editor window on the left contains an R Markdown document with the following content:

```
1. ---
2. title: "Untitled"
3. author: "wmj"
4. date: "2019/10/31"
5. output: html_document
6. ---
7.
8. ```{r setup, include=FALSE}
9. knitr::opts_chunk$set(echo = TRUE)
10. ```
11.
12. ## R Markdown
13.
14. This is an R Markdown document. Markdown is a simple formatting
    syntax for authoring HTML, PDF, and MS word documents. For more
    details on using R Markdown see http://rmarkdown.rstudio.com
```

The right-hand pane shows the 'Help' tab selected, displaying the documentation for the `glm` function. The title is 'Fitting Generalized Linear Models'. The description states: 'glm is used to fit generalized linear models, specified by giving a symbolic description of the linear predictor and a description of the error distribution.' The usage section shows the following code:

```
glm(formula, family = gaussian, data, weights, subset,
    na.action, start = NULL, etastart, mustart, offset,
    control = list(...), model = TRUE, method = "glm.fit",
    x = FALSE, y = TRUE, singular.ok = TRUE, contrasts = NULL, ...)
```

The 'Arguments' section lists the following parameters:

- formula**: an object of class "`formula`" (or one that can be coerced to that class): a symbolic description of the model to be fitted. The details of model specification are given under 'Details'.
- family**: a description of the error distribution and link function to be used in the model. For `glm` this can be a character string naming a family function, a family function or the result of a call to a family function. For `glm.fit` only the third option is supported. (See [family](#) for details of family functions.)
- data**: an optional data frame, list or environment (or object coercible by `as.data.frame` to a data frame) containing the variables in the model. If not found in data, the variables are taken from environment (formula), typically the environment from which `glm` is called.
- weights**: an optional vector of 'prior weights' to be used in the fitting process. Should be `NULL` or a numeric vector.
- subset**: an optional vector specifying a subset of observations to be used in the fitting process.

In the bottom left corner of the R Markdown editor, the text `?glm` is visible, indicating the command used to open the help page.



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
- 8. 取子集**
9. (R)Markdown



8.1 子集选取

对于有多个元素的对象，如何提取部分数据？

对象就是我们在计算机里新建了存储空间，好比一个盒子，我们可以往盒子里面装东西（**赋值**），可以查看里面的内容或者对里面的内容做计算（**函数**），也可以从盒子里取出部分东西（**子集选取**）。



8.2 向量取子集

有至少四种选取子集的方法

```
x <- c(1.1, 2.2, 3.3, 4.4, 5.5)
```

```
x[1]
```

```
## [1] 1.1
```

```
x[c(1, 3)]
```

```
## [1] 1.1 3.3
```

```
x[c(3, 1)]
```

```
## [1] 3.3 1.1
```

```
# 负整数：删除指定位置的元素
```

```
x[-2]
```

```
## [1] 1.1 3.3 4.4 5.5
```

```
x[c(-3, -4)]
```

```
## [1] 1.1 2.2 5.5
```

```
# 逻辑向量：将TRUE对应位置的元素提取出来
```

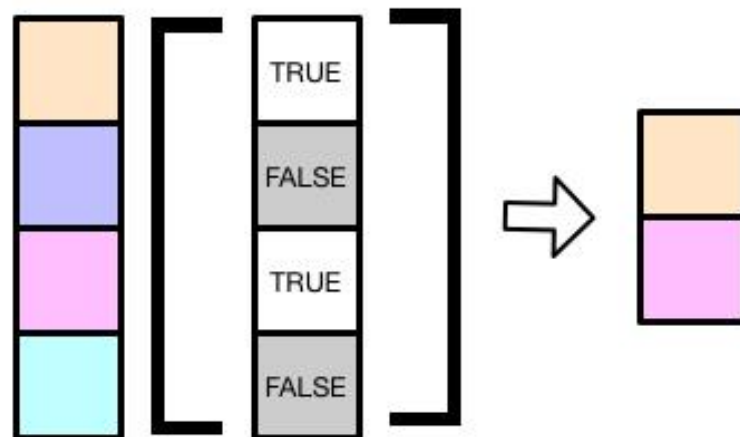
```
x[c(TRUE, FALSE, TRUE, FALSE, TRUE)]
```

```
## [1] 1.1 3.3 5.5
```

```
# 筛选出大于某个值的所有元素
```

```
x[x > 3]
```

```
## [1] 3.3 4.4 5.5
```





8.3 列表取子集

向量的子集仍然是向量，使用`[`提取列表的子集，总是返回列表



```
l <- list(
  "one" = c("a", "b", "c"),
  "two" = c(1:5),
  "three" = c(TRUE, FALSE)
)
l

## $one
## [1] "a" "b" "c"
##
## $two
## [1] 1 2 3 4 5
##
## $three
## [1] TRUE FALSE
```

```
# 使用位置索引
l[1]
## $one
## [1] "a" "b" "c"

# 可以使用元素名
l["one"]
## $one
## [1] "a" "b" "c"

# 提取列表某个元素的值，需要使用 [[
l[[1]]
l[["one"]]
l$one
## [1] "a" "b" "c"
```

- `[` 和 `[[` 的区别
- `x$y` 是 `x[["y"]]` 的简写



8.4 矩阵取子集

- 子矩阵
- 向量/元素

```
a <- matrix(1:9, nrow = 3)
```

```
a
```

```
##      [,1] [,2] [,3]  
## [1,]    1    4    7  
## [2,]    2    5    8  
## [3,]    3    6    9
```

```
a[1:2, 2:3]
```

```
##      [,1] [,2]  
## [1,]    4    7  
## [2,]    5    8
```

默认情况下，[会将获取的数据，以尽可能低的维度形式呈现

```
a[1, 1:2]
```

```
## [1] 1 4
```

```
a[1:2, ]
```

```
##      [,1] [,2] [,3]  
## [1,]    1    4    7  
## [2,]    2    5    8
```

```
a[, ] #?
```

```
a[] #?
```

```
a
```



8.5 数据框

数据框具有list和matrix的双重属性

- ◆ 当选取数据框的某几列的时候，可以和list一样，指定元素位置索引，比如df[1:2]选取前两列
- ◆ 也可以像矩阵一样，按照行和列的标识选取，比如df[1:3,]选取前三行的所有列



```
df <- data.frame(  
  x = 1:4,  
  y = 4:1,  
  z = c("a", "b", "c", "d")  
)  
df
```

```
df[1:2]  
##      x y  
## 1 1 4  
## 2 2 3  
## 3 3 2  
## 4 4 1
```

```
df[c("x", "z")] # or df[, c("x", "z")]  
##      x z  
## 1 1 a  
## 2 2 b  
## 3 3 c  
## 4 4 d  
  
df[["x"]]  
df$x  
## [1] 1 2 3 4
```



8.6 练习

➤ 如果x是一个矩阵，请问 $x[] \leftarrow 0$ 和 $x \leftarrow 0$ 有什么区别？

➤ 找出x向量中的偶数

```
x <- 1:10
```

```
x
```

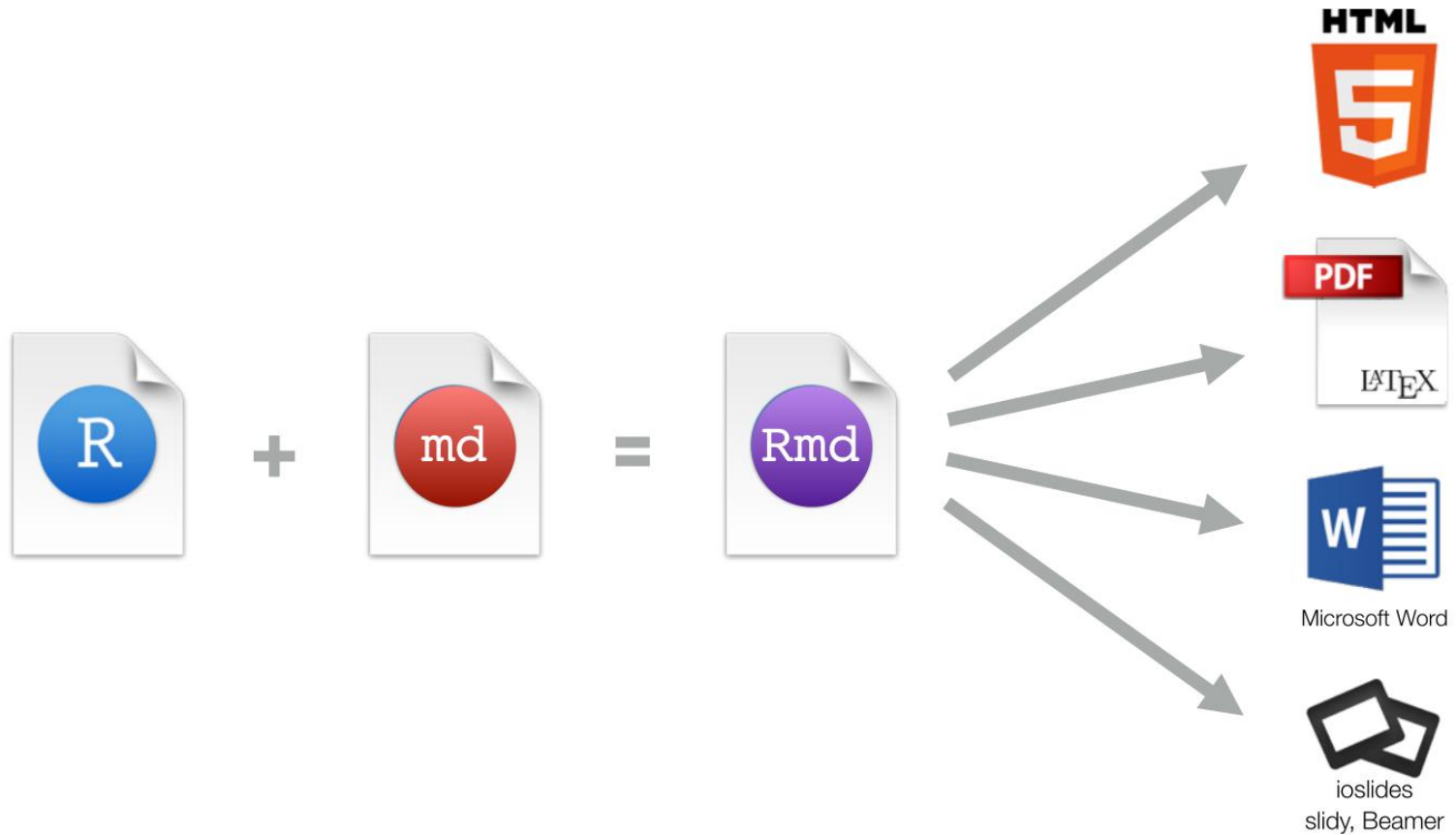


课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



9.1 什么是 RMarkdown





9.2 Markdown基本语法

<https://bookdown.org/yihui/rmarkdown/markdown-syntax.html>

如果想添加网页链接，可以用方括号和圆括号

```
[Download R](http://www.r-project.org/)
```



Download R

如果想制作简单表格，列与列之间用 | 分隔，表格的首行下面添加 -----

```
Table Header | Second Header
-----|-----
Cell 1, 1 | Cell 2, 1
Cell 1, 2 | Cell 2, 2
```



Table Header	Second Header
Cell 1, 1	Cell 2, 1
Cell 1, 2	Cell 2, 2



9.3 RMarkdown

The screenshot shows the RStudio interface with an R Markdown document titled 'Untitled1'. The document content is as follows:

```
1 ---
2 title: "notes"
3 author: "Kieran healy"
4 date: "9/13/2017"
5 output: html_document
6 ---
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for
15 authoring HTML, PDF, and MS Word documents. For more details on using R Markdown
16 see <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes
19 both content as well as the output of any embedded R code chunks within the
20 document. You can embed an R code chunk like this:
21
22 ```{r cars}
23 summary(cars)
24 ```
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 ```{r pressure, echo=FALSE}
31 plot(pressure)
32 ```
33
34 Note that the `echo = FALSE` parameter was added to the code chunk to prevent
35 printing of the R code that generated the plot.
```

Annotations on the screenshot include:

- A red circle around the **Knit** button in the top toolbar, with a green arrow pointing to it and the text "Process the whole document".
- A green bracket on the right side of the document, spanning from line 2 to line 5, with the text "Information about the document".
- A green bracket on the right side of the document, spanning from line 8 to line 10, with the text "Code chunk".
- A green bracket on the right side of the document, spanning from line 12 to line 35, with the text "Notes and discussion, with formatting instructions".
- A green circle with a play button icon on the right side of the document, next to line 9, with the text "Run just this chunk".
- A green circle with a play button icon on the right side of the document, next to line 23, with the text "Run all chunks up to this point".
- A green circle with a play button icon on the right side of the document, next to line 31, with the text "Set options for this code chunk".



9.3 RMarkdown

点击knit（下图中红色地方），选择想要输出的文档格式即可。下图是Rmd文档与渲染效果效果

The screenshot displays the RStudio interface with two main panes. The left pane shows the R Markdown source file 'chunks.Rmd'. The right pane shows the rendered HTML preview.

Left Pane (Source File):

```
1 R Code Chunks
2 =====
3
4 With R Markdown, you can insert R code
5 chunks including plots:
6
7 ```{r qplot, fig.width=4, fig.height=3,
8   message=FALSE}
9 # quick summary and plot
10 library(ggplot2)
11 summary(cars)
12 qplot(speed, dist, data=cars) +
13   geom_smooth()
```

The 'Knit HTML' button in the toolbar is circled in red.

Right Pane (Preview HTML):

R Code Chunks

With R Markdown, you can insert R code chunks including plots:

```
# quick summary and plot
library(ggplot2)
summary(cars)
```

##	speed	dist
## Min.	: 4.0	Min. : 2
## 1st Qu.:	:12.0	1st Qu.: 26
## Median :	:15.0	Median : 36
## Mean :	:15.4	Mean : 43
## 3rd Qu.:	:19.0	3rd Qu.: 56
## Max. :	:25.0	Max. : 120

```
qplot(speed, dist, data = cars) + geom_smooth()
```



rmarkdown :: CHEAT SHEET

RStudio® is a trademark of RStudio, PBC • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com • Learn more at rmarkdown.rstudio.com • rmarkdown 2.9.4 • Updated: 2021-08



课程大纲

1. 数据科学与R语言
2. R对象
3. R向量
4. R常见数据结构
5. 运算符与向量运算
6. R编程
7. 函数及应用
8. 取子集
9. (R)Markdown



课程作业 (RMarkdown生成和打印)

```
set.seed(学号)
```

同时上传到Gitee

```
x <- rnorm(20, mean = 60, sd = 15)
```

```
weight <- x
```

```
height <- rnorm(20, mean = 170, sd = 20)
```

- ✓ 自定义函数，根据方差的数学表达式，写出方差的计算函数，并与基础函数`var()`的结果对比

$$\text{Var}(x) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

- ✓ 自定义函数，它的作用是将输入的身高`height(cm)`与体重`weight(kg)`计算之后的BMI结果返回，BMI的计算公式为：

$$\text{BMI} = \frac{\text{weight}(kg)}{\text{height}(m)^2}$$

- ✓ 自定义函数，对于给定的向量 `x` 和阈值`threshold`（默认值60），求出`x`中所有大于该阈值的元素的均值



参考资料与致谢

1. 王敏杰, 《数据科学中的 R 语言》

<https://bookdown.org/wangminjie/R4DS/>

重度借用

2. Hadley, 《R for Data Science》

<https://r4ds.hadley.nz/>

经典

3. Posit R Cheatsheets

<https://posit.co/resources/cheatsheets/>

awesome

4. <https://carbon.now.sh/> 漂亮地秀代码